

## Mini Project 1: Vigenère Cipher and Password Cracker

(Due Sep 11, 2026)

Consider an encryption tool based on the Vigenère Cipher. It uses the password of the user as the key to encrypt the input message from the user, or decrypt the ciphertext to output the original message.

**Task 1: First, write a program to implement the Vigenère Cipher.**

- Encryption:  $E_K(m) = m + K \bmod 26$
- Decryption:  $D_K(m) = m - K \bmod 26$
- $m$  is the input message,  $K$  is the encryption/decryption key

Note:

1. Assume the plaintext message contains letters only. You don't need to distinguish between upper- and lower-case letters. That is, both "A" and "a" must be converted to "0" (or "1") in your program. If the input has both upper- and lower-case letters, treat them the same.
2. You don't need to deal with the space in the plaintext. If the input has spaces, remove them.
3. You don't need to check the validity of the input. Assume the input is always valid.

**Task 2: Next, implement a brute-force password cracker.**

Assume the plaintext message contains words in the *dictionary* provided. To assist the cracking, you're provided two additional information: an integer *keyLength* that denotes the length of the key, and an integer *firstWordLength* that denotes the length of the first word of the plaintext. Therefore, your password cracker will take three parameters:

- (1) a string of ciphertext
- (2) an integer *keyLength*
- (3) an integer *firstWordLength*

Use the **brute force strategy**:

1. Try every candidate key of length *keyLength*, ranging from all "A"s to all "Z"s.
2. For each candidate, generate the corresponding "plaintext" and check if the first "word" appears in the dictionary. The first word is the first *firstWordLength* letters.
3. If it does, display the plaintext and the key.
4. Do not stop at the first match, since the first match may be incorrect. Continue testing all the remaining keys.

Note:

- Remember to load the dictionary into memory before testing any keys.
- Efficiency is very important in evaluating each plaintext candidate. In some cases, a wrong key may still produce a valid first word, which means you could end up with multiple candidate plaintexts after testing all possible keys. This is acceptable. You can review the outputs and determine which key is truly correct.
- The key may not be a valid word, so you should not exploit the dictionary to guess the key.

**Task 3: Use your password cracker to decrypt the following messages. Decrypt at least 5 messages to receive full points.**

1. "MSOKKJCOSXOEKDTOSLGFWCMCHSUSGX"  
key length = 2; firstWordLength = 6
2. "PSPDYLOAFSGFREQKKPOERNIYVSDZSUOVGXSRIPWERDIPCFSDIQZIASEJVCGXAYBGYXFPSREKFMEX  
EBIYDGFKREOWGXEQXSXSKXGYRRRVMEKFFIPIWJSKFDJMBGCC"  
keyLength=3; firstWordLength = 7
3. "MTZHZEQKASVBDOWMWMKMNYIIHVWPEXJA"  
  
keyLength=4; firstWordLength = 10
4. "SQLIMXEEKSXMDOSBITOTYVECRDXSCRURZYPOHRG"  
keyLength=5; firstWordLength = 11
5. "LDWMEKPOPSWNOAVBIDHIPCEWAETVRVOAUPSINOVDIEDHCSELHCCPVHRPOHZUSERSFS"  
keyLength=6; firstWordLength = 9
6. "VVVLZWWPBWHZDKBTXLDCGOTGTGRWAQWZSDHEMXLBELUMO"  
keyLength=7; firstWordLength = 13

Note:

- Please record the time needed to decrypt each message.
- When key length increases, cracking becomes slow. Starting from the 4th message, cracking may take a long time. Optimize your code.
- **Discussion Question:** (1) discuss the efficiency of password cracking; (2) discuss the optimization technique(s) that you use and how it can improve cracking efficiency.

**Deliverables:** For Tasks 1 and 2, submit the code with readme about how to run it. For Task 3, submit the report with the decrypted messages and the answer to the discussion question.